



FACULTAD DE INGENIERÍA

INFORME INTEGRADO DE PRÁCTICA DE CAMPO

Programación de software para un ATM

Integrantes:

- RAMOS GUERRA, Jaime Eloy
- CHAVEZ PAUCAR, Cesar Luis
- ROJAS PALLIN, Joseph Daniel

Curso:

Estructura de Datos

Docente:

Juan Ricardo Tapia Carbajal

Lima – Perú

2026 – 1

PARTE I

INFORME DE PRÁCTICA DE CAMPO

1. Introducción

Este proyecto consiste en el desarrollo de una aplicación en C# que simula el funcionamiento de un cajero automático (ATM). El objetivo principal es proporcionar a los estudiantes un escenario práctico y real para aplicar estructuras de datos lineales, específicamente **pilas (Stacks)** y **colas (Queues)**, consolidando así los conocimientos teóricos adquiridos en clase. El sistema se diseñará bajo un enfoque intuitivo, garantizando una interfaz accesible para el usuario final mientras se resuelven problemas lógicos fundamentales mediante funciones básicas de transacciones bancarias.

2. Objetivos

2.1 Objetivo General

Implementar una aplicación de cajero automático (ATM) en C# con una interfaz intuitiva, con el fin de proporcionar a los estudiantes un entorno práctico y real que les permita aplicar e integrar conceptos fundamentales de estructuras de datos, como son listas, pilas y colas, así como demostrar cómo el software gestiona la memoria, el flujo de datos y los procesos de concurrencia en un entorno real que permita el acceso al usuario, control del dinero y la gestión de transacciones.

2.2 Objetivos Específicos

- Desarrollar una interfaz de usuario intuitiva.
- Diseño de clases del sistema en C#.
- Implementar las estructuras de datos en C#.
- Utilizar ramas para organizar el desarrollo.
- Validar el funcionamiento del sistema mediante pruebas múltiples.

3. Desarrollo de conceptos

3.1 Listas

Una Lista es una estructura de datos lineal y dinámica que almacena una colección secuencial de los elementos (nodos o datos), donde el orden de los elementos importa y cada uno de ellos ocupa una posición específica o índice. Esto a diferencia de los ya conocidos arreglos (arrays) las listas no tienen un tamaño fijo predefinido. Lo que significa que pueden crecer o encogerse dinámicamente en la memoria durante la ejecución del programa según requiera el sistema. Existen también diferentes tipos de listas:

- Listas Enlazadas Simples.
- Listas Doblemente enlazadas.
- Listas Enlazadas Circulares.
- Listas Enlazadas Doblemente Circulares.

Las listas enlazadas pueden ser implementadas en muchos lenguajes, algunas operaciones básicas de una lista:

- Recorrer los elementos hacia delante,
- Recorrer los elementos hacia atrás.
- Insertar un nuevo elemento al principio.
- Insertar un nuevo elemento al final.
- Insertar un nuevo elemento antes de otro.
- Insertar un elemento después de otro.
- Remover un elemento del principio.
- Remover un elemento que esta antes del otro.
- Remover un elemento que esta después del otro.
- Remover un elemento del principio.
- Remover un elemento del final.

3.2 Pilas

Es un método de estructuración de datos usando la forma LIFO (Último en entrar, primero en salir), que permite almacenar y recuperar datos.

Las operaciones que se pueden realizar con las pilas son:

- PUSH (Pila, elemento): Introduce un elemento en la pila, También se le conoce como por o insertar.
- POP (Pila): Elimina un elemento de la pila. También se le conoce como extraer o sacar.
- VACIA (Pila): Función booleana que indica si la pila es vacía o no.

3.3 Colas

Es una estructura de datos, caracterizada por una secuencia de elementos en la que la operación de inserción *push*, se realiza por un extremo y la extracción *pop* por el otro extremo. También se llama estructura FIFO (First In First Out), debido a que el primer elemento en entrar también será el primero en salir.

Tipos de colas:

- Cola Simple: Estructura lineal donde los elementos salen en el mismo orden en el que llegan.
- Cola circular: Representación lógica de una cola simple en un arreglo.
- Cola de prioridades: Estructura lineal en la cual los elementos se insertan en cualquier posición de la cola y se remueven únicamente por el frente.
- Cola doble (Bicola): Estructura lineal en la que los elementos se pueden añadir o quitar en cualquiera de los extremos de la cola. (Cola bidireccional).

Operaciones básicas en colas simples:

- Insertar: Almacena al final de la cola el elemento que se recibe como parámetro,
- Eliminar: Saca de la cola el elemento que se encuentra al frente.
- Vacía: Regresa un valor booleano indicando si la cola tiene o no elementos (true – si la cola está vacía, false- Si la cola tiene al menos un elemento).
- Llena: Regresa un valor booleano indicando si cola tiene espacio disponible para insertar nuevos elementos (true-si está llena y false si existen espacios disponibles).

4. Implementación de programa

A continuación, se presenta la secuencia del desarrollo del programa.

4.1 Gestión Dinámica de Clientes y Cuentas (Listas)

- Implementar una Lista Enlazada o List <T> en C# para almacenar, buscar y actualizar de forma dinámica la información de los usuarios (número de cuenta, PIN, saldo) sin las restricciones de tamaño fijo de un arreglo convencional.
- Desarrollar el algoritmo de validación de credenciales que recorra de manera eficiente la lista de cuentas para autenticar el ingreso del usuario al cajero.

4.2 Procesamiento Ordenado de Transacciones (Colas)

- Utilizar la estructura Cola (Queue <T>) para gestionar las solicitudes de retiro, depósito o consulta de los usuarios, garantizando que se procesen bajo el principio FIFO (First In, First Out), simulando fielmente la fila de espera en un servidor bancario.
- Diseñar un módulo de simulación de turnos donde las transacciones entrantes se encolen y se despachen secuencialmente, asegurando la integridad de las operaciones y evitando condiciones de carrera en el saldo de la base de datos simulada.

4.3 Registro Histórico y Reversión de Operaciones (Pilas)

- Implementar una Pila (Stack <T>) para registrar el historial de transacciones efectuadas por el usuario durante su sesión actual, permitiendo mostrar los movimientos desde el más reciente al más antiguo (principio LIFO: Last In, First Out).
- Desarrollar una función de "Deshacer última acción" o reversión de transacciones pendientes utilizando la pila, facilitando el retorno al estado anterior de la cuenta en caso de cancelación por parte del usuario.

4.4 Modularidad y Buenas Prácticas de Programación

- Aplicar los principios de la Programación Orientada a Objetos (POO) mediante la creación de clases específicas (Cliente, Transacción, ATM) y la encapsulación de los métodos de manipulación de las estructuras de datos.
- Garantizar la estabilidad del sistema mediante el manejo de excepciones (try-catch) para controlar errores comunes en cajeros, tales como fondos insuficientes, intentos de PIN inválidos o desbordamientos en las estructuras.

5. Interfaz de usuario

5.1 Interfaz de usuario

Muestra en pantalla los siguientes mensajes, un mensaje de bienvenida al usuario solicita su número de identificación y su contraseña los datos serán ingresados por teclado, el usuario tendrá 3 intentos como máximo de lo contrario, mostrará el mensaje “Cuenta bloqueada”, una vez que ingrese correctamente su clave se mostrará los siguientes mensajes: “Menú principal”, “1. Consulta de saldo y movimientos”, “2. Retiro de efectivo”, “3. Depósito de fondos”, “4.

Pago de tarjeta de crédito”, “5. Cambio de clave”, 6. “SALIR”, de igual forma las opciones deseadas serán ingresadas mediante teclado.

```
*=== BIENVENIDO AL ATM ===*  
Ingrese número de cuenta: 19990  
Ingrese su clave (PIN): 0000  
  
Clave correcta. Ingresando...  
  
=== Menú principal ===  
1. Consulta de saldo y movimientos  
2. Retiro de efectivo  
3. Depósito de fondos  
4. Pago de tarjeta de crédito  
5. Cambio de clave  
6. SALIR  
Seleccione una opción: |
```

Ilustración 1: Menú Principal.

Si el usuario selecciona la **opción 1**, se mostrará el saldo de la cuenta.

```
--- CONSULTA ---  
Saldo Actual: $2000  
  
--- Últimos Movimientos ---  
No registra movimientos en esta sesión.
```

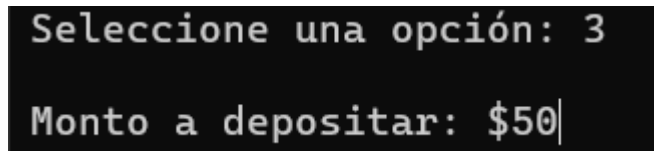
Ilustración 2: Consulta de movimientos.

Si el usuario selecciona la **opción 2**, se muestra las siguientes opciones: “Menú de retiro”, “1. S/ 20”, “2. S/ 40”, “3. S/ 60”, “4. S/ 100”, “5. Ingrese Monto”, “6. Cancelar retiro”, “Elija un monto de retiro”.

```
Monto a retirar: $100  
Retiro exitoso. Retire su efectivo.
```

Ilustración 3: Menú de retiro.

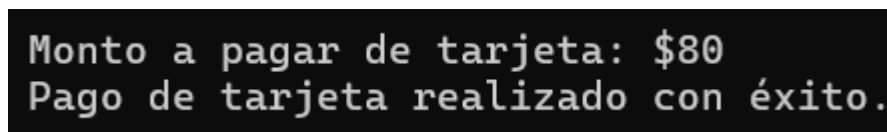
La **opción 3** del menú principal se utilizará para hacer un depósito de dinero, para ello en la pantalla principal se visualizará un mensaje que pide al usuario que ingrese la opción: “1. Ingrese Monto”, el usuario debe ingresar el monto de depósito o que escriba “2. Salir” para cancelar la transacción y nuevamente volverá al menú principal.



```
Seleccione una opción: 3
Monto a depositar: $50|
```

Ilustración 4:Manú de deposito

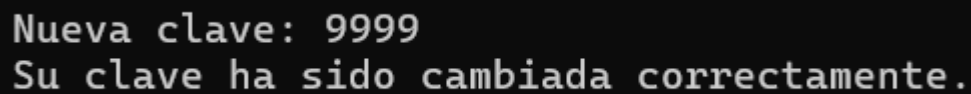
En cuanto a la **opción 4** del menú principal, este será direccionado a la opción de pagar tus tarjetas de crédito ingresando la opción: “1. Ingrese monto a pagar”, para cancelar simplemente se ingresa la opción: “2. Salir”.



```
Monto a pagar de tarjeta: $80
Pago de tarjeta realizado con éxito.
```

Ilustración 5:Pago de tarjeta de crédito.

Así mismo la **opción 5** permite el cambio de clave y para finalizar la ejecución del programa se tiene la **opción 6**



```
Nueva clave: 9999
Su clave ha sido cambiada correctamente.
```

Ilustración 6: Cambio de clave

7. Conclusiones

En conclusión, las listas, pilas y colas son estructuras de datos que están hechas para facilitar la programación ya que, son fáciles de utilizar y podemos utilizarlas para desarrollar una cantidad de operaciones como en este proyecto un programa aplicable a un cajero automático.

8. Bibliografía

Deitel, P. y Deitel, H. (2017). Java How to program. (11° ed.). New York, USA: Pearson